

Achtung!

20260611

Wichtiger Hinweis zur Vorlesung (SVM):

Sollten Sie bemerken, dass die mathematische Gleichung aus der alten Version

$$-1.75x + 1.55y + 2.66 = 0 \quad (\text{FALSCH!})$$

optisch überhaupt nicht zu der gezeichneten Linie im Plot passt, liegt das an einem Fehler im ursprünglichen Skript $\Rightarrow$ (Blöcke 4 und 5).

$\Rightarrow$ Dort wurden der Normalenvektor (senkrecht zur Linie) und der Richtungsvektor (parallel entlang der Linie) bei der Berechnung von $ax + by + c$ vertauscht. Dies führte zu einer fehlerhaften 90° Drehung in der Notebook-Ausgabe!

Die mathematische einzig richtige Gleichung lautet:

$$1.55x + 1.75y - 9.45 = 0$$

RICHTIG!

WICHTIG: Das Python Notebook im GitHub wurde aktualisiert!

HAFTUNGSAUSSCHLUSS - Hinweis zur Prüfung

Die Studierenden sollten davon ausgehen, dass sowohl der in den Vorlesungen behandelte Stoff als auch die auf GitHub hochgeladenen Materialien für die Prüfungsvorbereitung relevant sind.

* Dies umfasst jeglichen Code, der hochgeladen, aber nicht in den Vorlesungen behandelt wurde.

AKIB2 - 20260610

K-Means Clustering:

⇒ Setup und Datensimulation

```
import numpy as np # Für numerische Operationen
import matplotlib.pyplot as plt # Für Visualisierung

# ----- Datensimulation: Fabrikleistung -----
# Wir simulieren 10 Produktionsanlagen mit zwei Metriken:
# - Durchsatz (Stück pro Stunde)
# - Kosten pro Einheit (Euro)

np.random.seed(0) # Für Reproduzierbarkeit

durchsatz = np.array([100, 110, 95, 130, 105, 250, 260, 240, 270, 255]) # Simulierter Durchsatz
kosten = np.array([5.0, 4.8, 5.5, 4.5, 5.2, 2.0, 1.8, 2.1, 1.7, 2.2]) # Simulierte Stückkosten

# Kombinieren der Daten in ein zweidimensionales Array (10x2)
data = np.column_stack((durchsatz, kosten))
```

→ `np.random.seed(0)`: Fixiert den Zufall, damit die initialen Zentren reproduzierbar bleiben.

→ `np.column_stack`: Transformiert die beiden 1D-Vektoren in eine gemeinsame $N \times 2$ Matrix. Jede Maschine i wird als ein 2-dimensionaler Vektor im euklidischen Raum definiert.

⇒ Funktion: Distanzberechnung

```
def compute_distances(data, centroids):  
    """Berechnet die euklidische Distanz jedes Punktes zu allen Zentren."""  
    distances = np.zeros((data.shape[0], len(centroids)))  
    for i, centroid in enumerate(centroids):  
        distances[:, i] = np.linalg.norm(data - centroid, axis=1)  
    return distances
```

- `np.zeros((data.shape[0], len(centroids)))`: initialisiert eine leere Distanzmatrix der Dimension $10 \times K$, um die Abstände zu speichern.
- `np.linalg.norm(..., axis=1)`: Berechnet zeilenweise die Euklidische Distanz (L_2 -Norm) zwischen allen Datenpunkten x und den aktuellen Zentroiden z_i

$$d = \|x - z_i\|_2 = \sqrt{(x_1 - z_{i,1})^2 + (x_2 - z_{i,2})^2}$$

⇒ Funktion: Iterations-visualisierung

```
def plot_iteration(data, centroids, labels, new_centroids, iteration):  
    """Visualisiert einen k-Means-Schritt."""  
    colors = ['red' if label == 0 else 'blue' for label in labels]  
  
    plt.figure(figsize=(8, 6))  
    plt.scatter(data[:, 0], data[:, 1], c=colors, s=100, label='Maschinen')  
    plt.scatter(centroids[:, 0], centroids[:, 1], c='black', marker='x', s=200, label='Zentren vorher')  
    plt.scatter(new_centroids[:, 0], new_centroids[:, 1], c='green', marker='P', s=200, label='Zentren neu')  
  
    for i, (x, y) in enumerate(data):  
        plt.text(x + 1, y, f"M{i+1}", fontsize=9)  
  
    plt.title(f"k-Means Iteration {iteration}")  
    plt.xlabel("Durchsatz (Stück/Stunde)")  
    plt.ylabel("Kosten pro Einheit (€)")  
    plt.legend()  
    plt.grid(True)  
    plt.show()
```

- `colors`: Beschreibt die Farbe, basierend auf der Gruppenzugehörigkeit
 $G_0 = \text{Rot}$, $G_1 = \text{Blau}$
- `plt.scatter`: Plottet die Maschinen x , die alten Zentroiden $z^{(t)}$ (schwarzes x) und die neu berechneten Zentroiden $z^{(t+1)}$ (grünes P), um das Wandern visuell greifbar zu machen.
- `plt.text`: Schreibt das Label der Maschine (M_1 bis M_{10}) direkt neben den Datenpunkt auf dem Graphen.

⇒ Funktion: k-Means Kern-Algorithmus

```
def k_means_step_by_step(data, k=2, max_iter=10):  
    """Führt k-Means durch und visualisiert jede Iteration."""  
    # Zufällige Initialisierung  
    centroids = data[np.random.choice(len(data), k, replace=False)]  
  
    for iteration in range(1, max_iter + 1):  
        # Schritt 1: Distanzen berechnen und Clusterzuweisung  
        distances = compute_distances(data, centroids)  
        labels = np.argmin(distances, axis=1)  
  
        # Schritt 2: Zentren aktualisieren  
        new_centroids = np.array([data[labels == i].mean(axis=0) for i in range(k)])  
  
        # Visualisierung dieser Iteration  
        plot_iteration(data, centroids, labels, new_centroids, iteration)  
  
        # Abbruchbedingung: Zentren haben sich nicht verändert  
        if np.allclose(centroids, new_centroids):  
            break  
        centroids = new_centroids  
  
    return labels, centroids
```

- `np.random.choice` (Schritt 0 + 1): Wählt k eindeutige Zeilenindizes aus den Daten als initiale Zentroide - Koordinaten z_k .
- `np.argmin(distances, axis=1)` (Schritt 4): Ordnet jeden Punkt der Gruppe G zu, deren Zentroid z am nächsten liegt:

$$\operatorname{argmin}_i \|x - z_i\|^2$$

- `data[labels == i].mean(axis=0)` (Schritt 2): Berechnet die neuen Schwerpunkt (Arithmetisches Mittel) für alle Vektoren, die sich in Gruppe G_i befinden;

$$z_i = \frac{1}{|G_i|} \sum_{x_j \in G_i} x_j$$

- `np.allclose`: Prüft die Mathematische Konvergenz.
⇒ Wenn $z^{(t+1)} = z^{(t)}$, bricht die Schleife vorzeitig ab.

⇒ Modellausführung und Ergebnisausgabe

```
# ----- Ausführung -----  
labels, final_centroids = k_means_step_by_step(data, k=2)  
  
# ----- Ausgabe der finalen Zuordnung -----  
for i, (d, c, label) in enumerate(zip(durchsatz, kosten, labels)):  
    print(f"Maschine {i+1}: Durchsatz={d}, Kosten={c} → Cluster {label}")
```

- **k_means_step_by_step()**: Startet die Berechnungen und gibt die finalen Gruppen-IDs (Labels) sowie die optimierten Zentroiden zurück.
- **for... zip(...)**: Iteriert simultan über die Rohdaten und die gelernten Gruppen-IDs. Jede Maschine ist nun eindeutig einer Gruppe zugeordnet
G₀ oder G₁.

K Nearest Neighbors (KNN)

Entsprechend den Vorlesungsnotizen führen wir neue Datenpunkte live auf eine nachbarschaftsbasierte Mehrheitsentscheidung in bestehenden Klassen über. Da KNN ein **Lazy-Algorithmus** ist, entfällt eine dedizierte Trainingsphase; die gesamte Rechenarbeit wird während der Klassifikationsfrage erbracht.

Manueller KNN-Algorithmus (Schritt für Schritt):

⇒ Datensimulation und Raum-Definition

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# ----- 1. Simulierte Beispieldaten -----
# Zwei Merkmale: Gewicht (Merkmal1) und Geschwindigkeit (Merkmal2)
merkmal1 = np.array([55, 80, 45, 70, 65, 50, 85, 60, 75, 90]) # Gewicht
merkmal2 = np.array([20, 35, 15, 30, 25, 18, 40, 22, 28, 38]) # Geschwindigkeit
klassen = np.array([0, 1, 0, 1, 1, 0, 1, 0, 1, 1]) # Zielklassen: 0 oder 1

# Kombiniere die Merkmale zu einem Datensatz
daten = np.column_stack((merkmal1, merkmal2))
```

- **np.column_stack**: Kombiniert die Vektoren **merkmal1** (Gewicht) und **merkmal2** (Geschwindigkeit) zu einer 10×2 Matrix. Jedes Objekt wird als Vektor

$$x_i \in \mathbb{R}^2$$

in euklidischen Raum verortet.

- Hinweis zum Wertebereich: Das Gewicht (45-90) liegt auf einer völlig anderen Skala als die Geschwindigkeit (15-40). Eine Normalisierung ist zwingend erforderlich, da das Gewicht sonst die Distanzberechnung dominiert.

⇒ Schritt 1: Min-Max Normalisierung

```
# ----- 2. Normalisierung mit Min-Max -----  
# Wir bringen die Daten auf einen Bereich von 0 bis 1  
  
min_vals = daten.min(axis=0)  
max_vals = daten.max(axis=0)  
daten_norm = (daten - min_vals) / (max_vals - min_vals)  
  
# Anfragepunkt (z.B. ein neuer Roboter mit Gewicht 65 und Geschwindigkeit 29)  
anfrage = np.array([65, 29])  
anfrage_norm = (anfrage - min_vals) / (max_vals - min_vals)
```

- **daten_norm / anfrage_norm**: Setzt die exakte Min-Max Scaler Formel aus den handschriftlichen Notizen um.
⇒ Alle Features werden linear in das Intervall $[0, 1]$ gepresst:

$$x_i^* = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

- **anfrage**: Der Vektor x_{new} repräsentiert das neue, unbekannte Objekt, dessen Klassenzugehörigkeit $(0, 1)$ bestimmt werden soll.

⇒ Schritt 2 + 3: Distanzberechnung und Sortierung

```
# ----- 3. Manuelle KNN-Funktion über k = 1...9 -----  
ks = range(1, 10)  
klassen_ids = sorted(set(klassen))  
wahrscheinlichkeiten = {c: [] for c in klassen_ids}  
  
# Berechne die Distanz zu allen Punkten (euklidisch)  
distanzen = [np.linalg.norm(p - anfrage_norm) for p in daten_norm]  
paare = list(zip(distanzen, klassen))  
paare.sort(key=lambda x: x[0]) # Sortiere nach Distanz (nächste Nachbarn zuerst)
```

- **np.linalg.norm(p - anfrage_norm)**: Berechnet die euklidische Distanz zwischen allen normierten Trainingspunkten x_i^* und den normierten Anfragevektor x_{new}^* .

$$d = \|x_i^* - x_{\text{new}}^*\|_2 = \sqrt{(x_{i,1}^* - x_{\text{new},1}^*)^2 + (x_{i,2}^* - x_{\text{new},2}^*)^2}$$

- **paare.sort**: Setzt die Vorlesungsanweisung "Abstände werden aufsteigend geordnet" um. Die nächsten Nachbarn wandern an die vordesten Indizes der List.

⇒ Schritt 4: Wahrscheinlichkeiten der Nachbarschaften

```
# Berechne für jedes k die Klassenwahrscheinlichkeiten
for k in ks:
    nachbarn = paare[:k]
    labels = [label for _, label in nachbarn]
    zaehler = Counter(labels)
    for c in klassen_ids:
        anteil = zaehler[c] / k if c in zaehler else 0.0
        wahrscheinlichkeiten[c].append(anteil)

print("Distanzen", distanzen)
print("wahrscheinlichkeiten", wahrscheinlichkeiten)
```

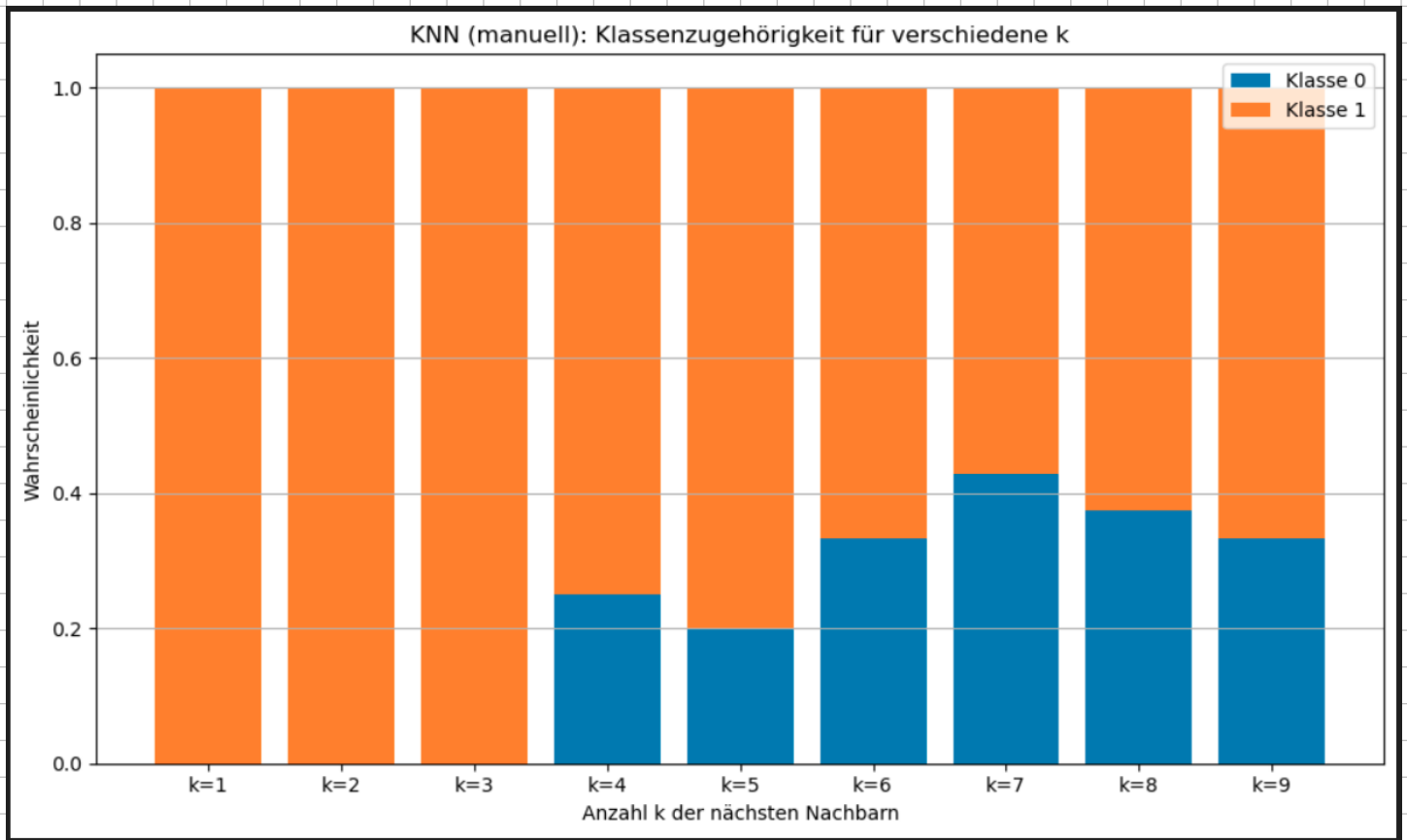
- **paare[:k]**: Schneidet das Fenster der k nächsten Nachbarn aus.
- **anteil**: Berechnet die relative Klassenhäufigkeit (p) innerhalb des aktuellen Nachbarschaftsradius K :
$$p(X_{\text{new}} \in \text{Klasse } c) = \frac{\# \text{ Nachbarn aus Klasse } c}{K}$$
- Bezug zur Skizze: Dies berechnet exakt die Brüche aus der Vorlesung
z.B. $p = \frac{2}{3}$ für Klasse N bei $K = 3$.

Das gestapelte Balkendiagramm visualisiert diese Verteilung über die gesamte Laufweite von

$$K = 1 \dots 9.$$

Das Minimum der Fehlerkurve bestimmt das optimale K .

⇒ Visualisierung



Scikit-Learn Implementierung

⇒ Automatisierte Skalierung und Modell-Fit

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier

# Skalieren die Daten mit sklearn
scaler = MinMaxScaler()
daten_sklearn = scaler.fit_transform(daten)
anfrage_sklearn = scaler.transform([anfrage])

# Liste der Wahrscheinlichkeiten pro k
wahrscheinlichkeiten_sklearn = []

# Für jedes k trainieren und Wahrscheinlichkeiten berechnen
for k in ks:
    modell = KNeighborsClassifier(n_neighbors=k)
    modell.fit(daten_sklearn, klassen)
    probs = modell.predict_proba(anfrage_sklearn)[0]
    wahrscheinlichkeiten_sklearn.append(probs)

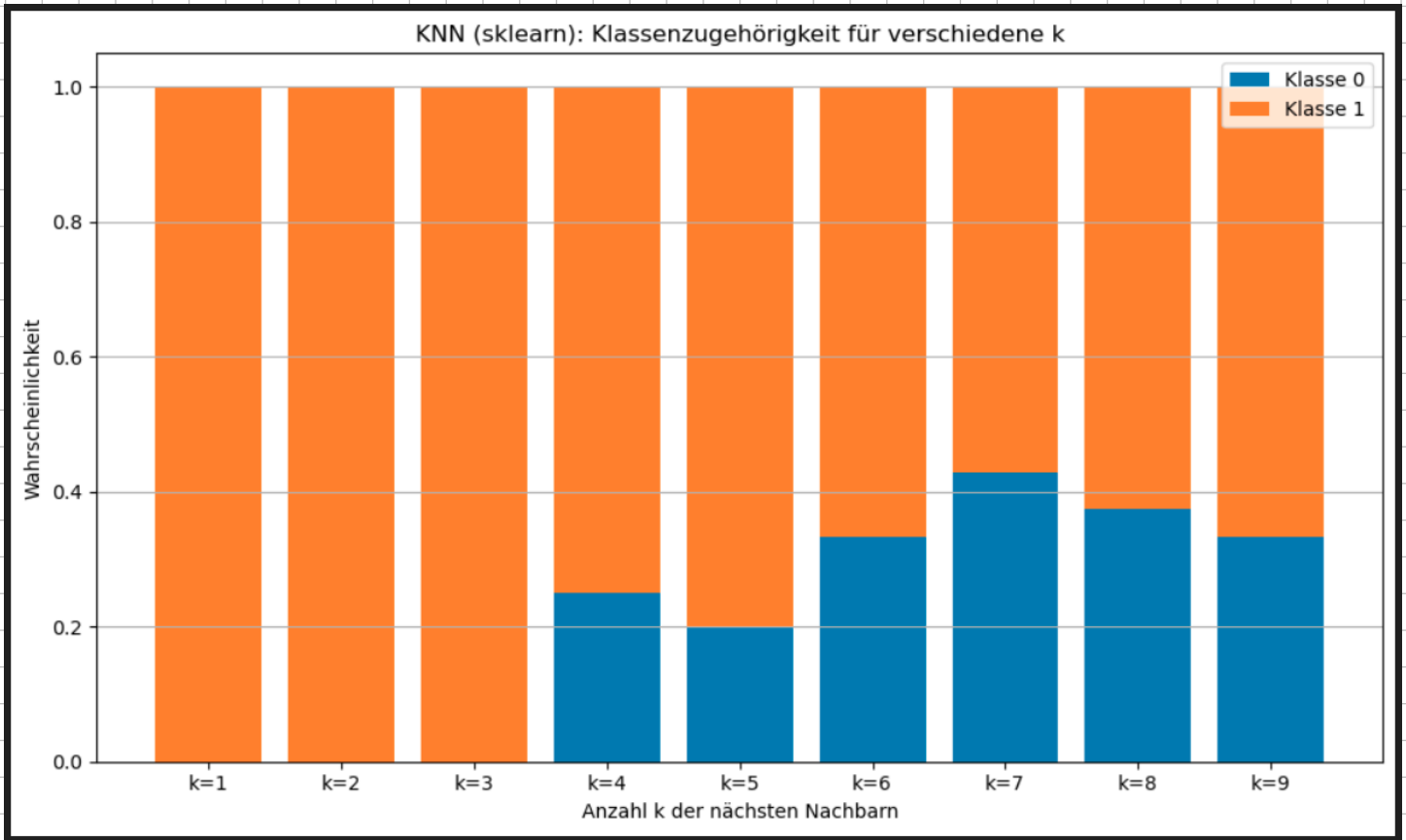
# Transponieren für gestapelte Balken
wahrscheinlichkeiten_sklearn = np.array(wahrscheinlichkeiten_sklearn).T
```

- **MinMaxScaler()**: Führt das fit-Transform auf den Trainingsdaten aus. Es ermittelt automatisch $X_{\min}$ sowie $X_{\max}$ und skaliert die Matrix eigenständig. **transform** wendet dieselbe Abbildung auf die neue Anfrage an.
- **KNeighborsClassifier(n_neighbors=k)**: Initialisiert das KNN-Modell aus der Bibliothek für den jeweiligen Nachbarschaftswert k .
- **predict_proba**: Liefert direkt das Array der berechneten Klassenwahrscheinlichkeiten zurück.
- **np.array(...).T**: Transponiert das Ergebniss-Array, um die Datenstruktur perfekt für das anschließende **plt.bar**-Plotting auszurichten.

* Transponieren (.T) $v = (a, b, c)$

$$v^T = \begin{pmatrix} a \\ b \\ c \end{pmatrix}$$

⇒ Visualisierung



Support Vector Machine (SVM)

⇒ Block 1: Datenerstellung im euklidischen Raum

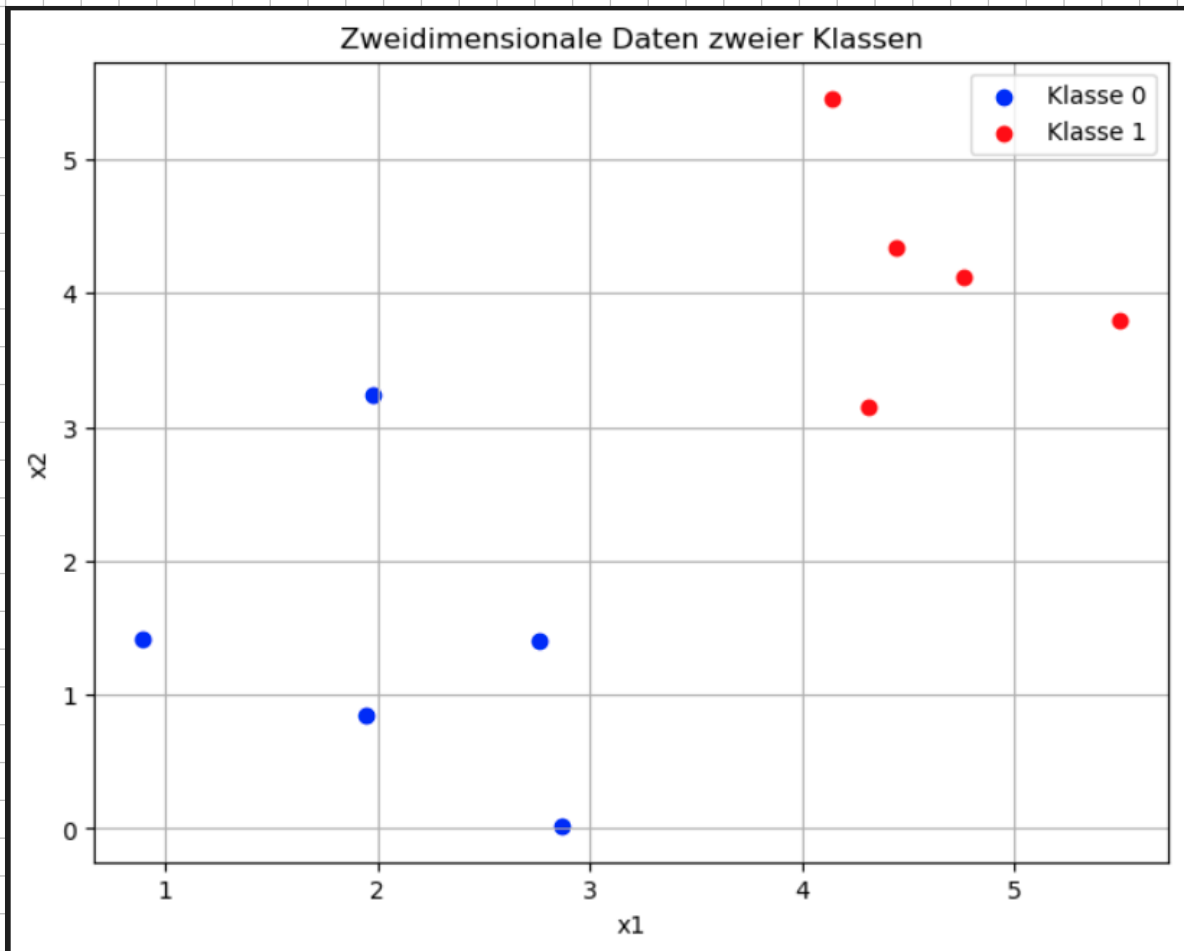
```
# ----- 1. Datenerstellung -----  
np.random.seed(0) # Reproduzierbarkeit  
klasse_0 = np.random.randn(5, 2) + np.array([1, 1]) # Klasse 0 nahe Punkt (1,1)  
klasse_1 = np.random.randn(5, 2) + np.array([4, 4]) # Klasse 1 nahe Punkt (4,4)  
  
data = np.vstack((klasse_0, klasse_1)) # Gesamtdaten (10x2)  
labels = np.array([0]*5 + [1]*5) # Zielklassen
```

- `np.random.randn(5, 2)` Erzeugt jeweils 5 zufällige 2D Punkte nach einer Standardnormalverteilung.
- `np.array([...])`: Verschiebt die Zentren der Punktwolken. Klasse-0 gruppiert sich um die Koordinaten (1, 1), während Klasse-1 um (4, 4) zentriert wird. Dadurch entsteht ein linear separierbarer Raum $\mathbb{R}^2$.
- `np.vstack` Stapelt die beiden Matrizen vertikal zu einer kombinierten 10×2 Gesamtmatrix (data).

⇒ Block 2: Visualisierung der Rohdaten

```
# ----- 2. Visualisierung der Daten -----  
plt.figure(figsize=(8, 6))  
plt.scatter(klasse_0[:, 0], klasse_0[:, 1], color='blue', label='Klasse 0')  
plt.scatter(klasse_1[:, 0], klasse_1[:, 1], color='red', label='Klasse 1')  
plt.title('Zweidimensionale Daten zweier Klassen')  
plt.xlabel('x1')  
plt.ylabel('x2')  
plt.grid(True)  
plt.legend()  
plt.show()
```

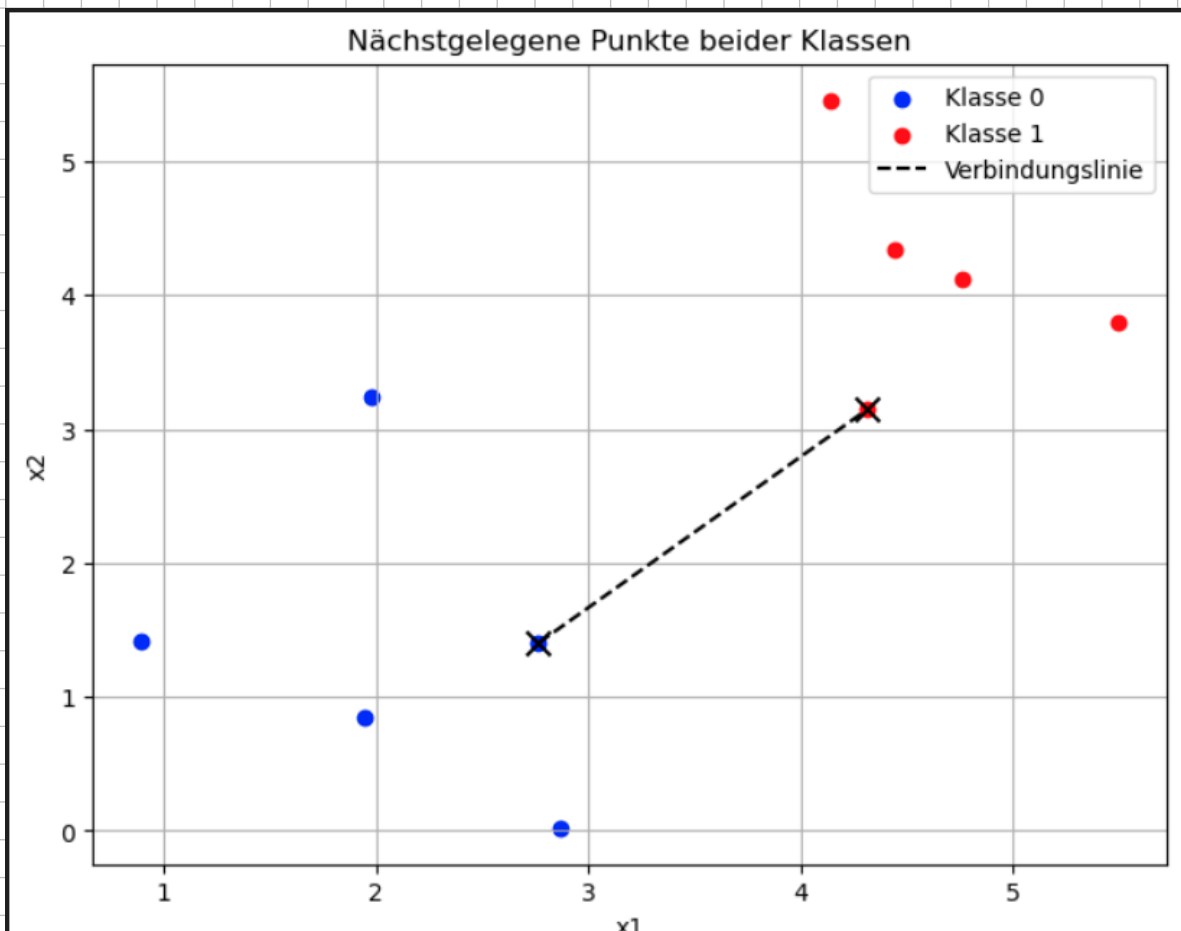
Schritt 1 der Vorlesung (Visualisierung): Bevor Berechnungen starten, wird die Punktverteilung geplottet. Wir sehen visuell zwei klar voneinander getrennte Cluster (Blau v. Rot). Das Ziel ist es nun, die exakte sauberste Trennlinie zwischen diesen Wolken zu finden.



=> Block 3: Bestimmung der Support - Vektoren

```
# ----- 3. Finden der nächstgelegenen Punkte -----  
min_dist = float('inf')  
punkt_a, punkt_b = None, None  
for p0 in klasse_0:  
    for p1 in klasse_1:  
        dist = np.linalg.norm(p0 - p1)  
        if dist < min_dist:  
            min_dist = dist  
            punkt_a, punkt_b = p0, p1
```

- **for p1 in klasse_1:** Eine Brute-Force-Suche, die den euklidischen Abstand zwischen jedem Blauen Punkt p_0 und jedem roten Punkt p_1 misst.
- **$\text{np.linalg.norm}(p_0 - p_1)$:** L_2 -Norm (siehe oben)



⇒ Block 4: Geometrische Konstruktion der Linienparameter

```
# ----- 4. Berechnung der Trennlinien -----  
mittelpunkt = (punkt_a + punkt_b) / 2  
normalvektor = punkt_b - punkt_a  
richtung = np.array([-normalvektor[1], normalvektor[0]])  
  
# Funktion zur Erzeugung einer Linie entlang eines Vektors  
def linie_durch_punkt_und_richtung(startpunkt, richtung, farbe, label):  
    linie_x, linie_y = [], []  
    for t in [-3, 3]:  
        punkt = startpunkt + t * richtung / np.linalg.norm(richtung)  
        linie_x.append(punkt[0])  
        linie_y.append(punkt[1])  
    plt.plot(linie_x, linie_y, farbe, label=label)
```

- **Mittelpunkt**: Berechnet die exakt geometrische Mittelpunkt x_m zwischen den beiden support-Vektoren. Hier muss die finale Trennlinie exakt hindurchlaufen.
- **richtung**: Der Verbindungsvektor $r = p_b - p_a$.
- **normalvektor**: Erzeugt der Orthogonalen Normalenvektor $n = [-r_y, r_x]$ durch vertauschen der Komponenten und Negation.
- **linie_durch_punkt_und_richtung()**: Normiert den Richtungsvektor mittels np.linalg.norm auf die Länge 1 und skaliert ihn auf Parameter t , nach links und rechts, um die Geraden sauber zeichnen zu können.

⇒ Block 5: Visualisierung von Trennlinie und Margins

```
# ----- 5. Visualisierung aller drei Trennlinien -----
plt.figure(figsize=(8, 6))
plt.scatter(klasse_0[:, 0], klasse_0[:, 1], color='blue', label='Klasse 0')
plt.scatter(klasse_1[:, 0], klasse_1[:, 1], color='red', label='Klasse 1')
plt.scatter([punkt_a[0], punkt_b[0]], [punkt_a[1], punkt_b[1]], color='black', marker='x', s=100)

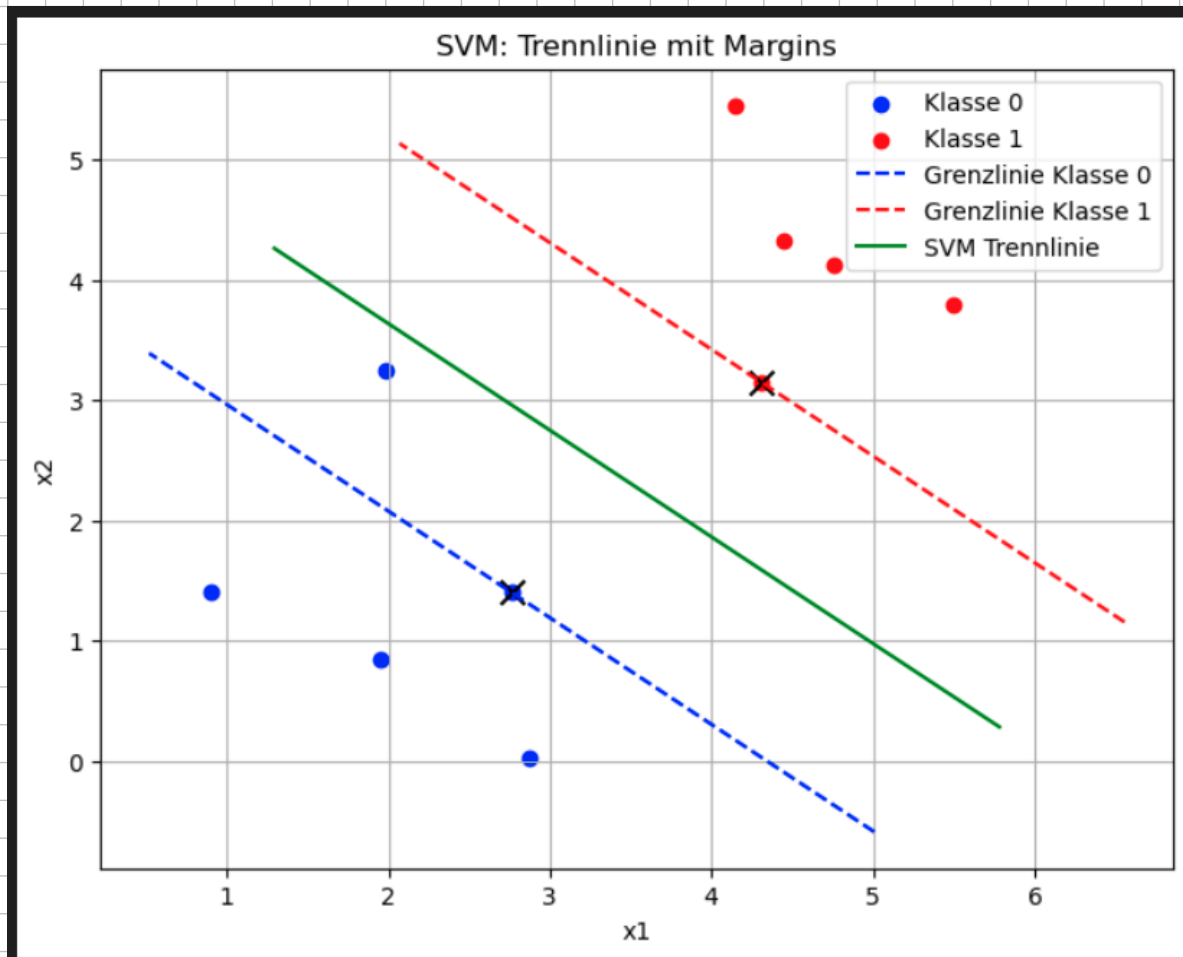
# Linie durch Punkt A (Support-Vektor Klasse 0)
linie_durch_punkt_und_richtung(punkt_a, normalvektor, 'b--', 'Grenzlinie Klasse 0')

# Linie durch Punkt B (Support-Vektor Klasse 1)
linie_durch_punkt_und_richtung(punkt_b, normalvektor, 'r--', 'Grenzlinie Klasse 1')

# Mittlere Trennlinie (SVM Margin)
linie_durch_punkt_und_richtung(mittelpunkt, normalvektor, 'g-', 'SVM Trennlinie')

plt.title('SVM: Trennlinie mit Margins')
plt.xlabel('x1')
plt.ylabel('x2')
plt.legend()
plt.grid(True)
plt.show()
```

- **b- / r-** Zeichnet die gestrichelten äußeren Begrenzungslinien direkt auf den Support-Vektoren. Der senkrechte Abstand zwischen diesen beiden Linien ist die max Breite der Straße (MARGIN).
- **g-** Zeichnet die durchgezogene grüne Mittellinie, die den exakt maximalen und identischen Abstand zu den kritische Punkten beider Klassen aufweist.



⇒ Block 6: Berechnung der Koordinatenform (Geradengleichung)

```
# ----- 6. Gleichung der Trennlinie -----  
a, b = normalvektor  
c = -(a * mittelpunkt[0] + b * mittelpunkt[1])  
  
print("Gleichung der SVM-Trennlinie:")  
print(f"{a:.2f}*x + {b:.2f}*y + {c:.2f} = 0")
```

- Überführung in die mathematische Normalform: Eine Linie in 2D-Raum kann über das Skalarprodukt aus Normalenvektor und Ortsvektor beschrieben werden:

$$w \cdot (x - x_a) = 0$$

- Ausmultipliziert ergibt das:

$$w_x x + w_y y - (w_x x_a + w_y y_a) = 0.$$

- Das Skript weist den Komponenten exakt wie die Variablen zu:

$$a = w_x, \quad b = w_y$$

und berechnet das absolute Glied $c = -w_x x_a$.

Das Terminal gibt uns die mathematische Gleichung in der impliziten Form aus:

$$ax + by + c = 0.$$